

Analizando servidores MySQL com o PMM

um passo a passo

▶ **Fernando Laudares Camargos**
Principal Architect



MySQL BR Conf 2026





Percona Monitoring and Management

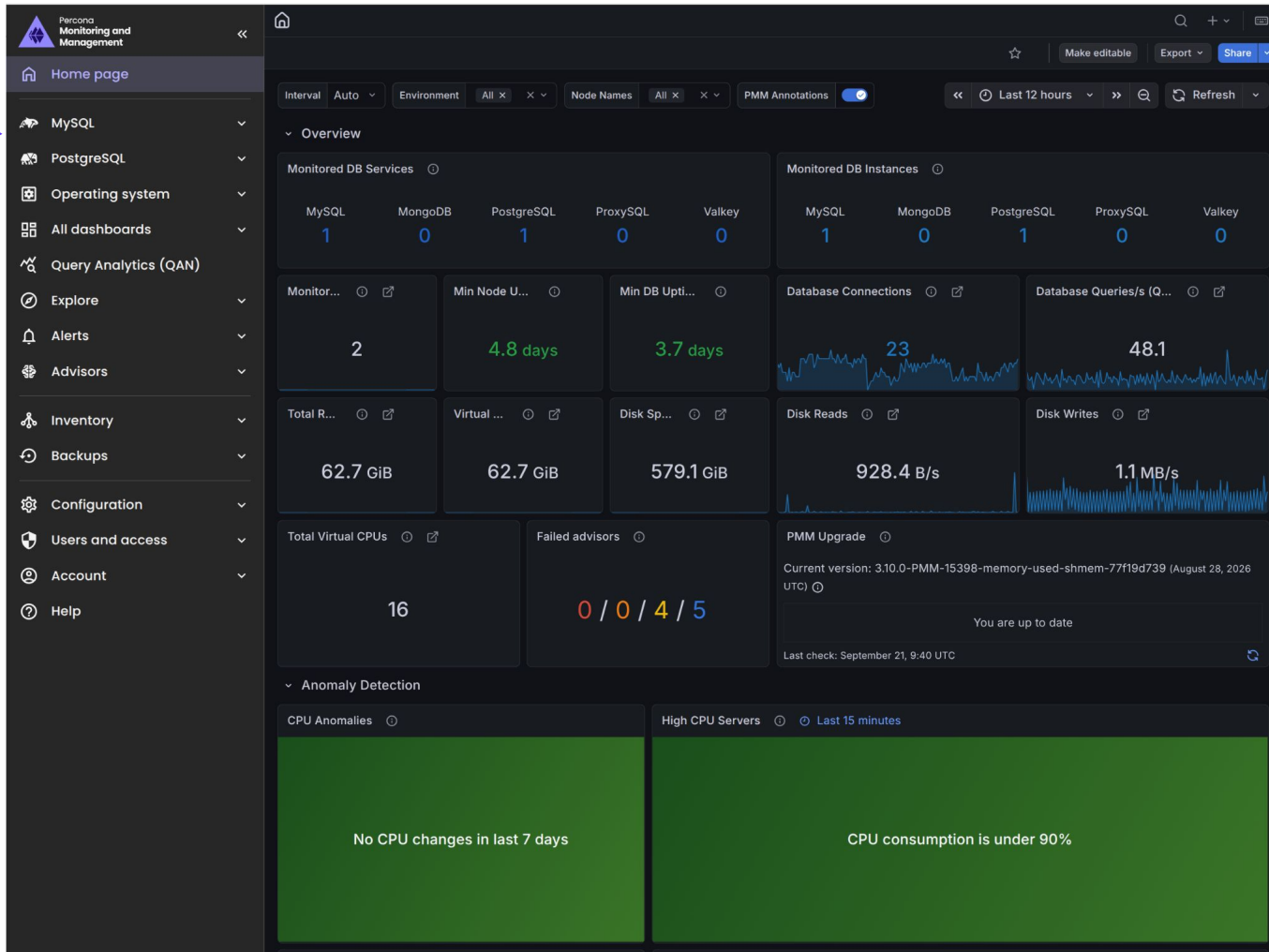
Your single pane of glass

Email or username

Password

Log in

[Forgot your password?](#)

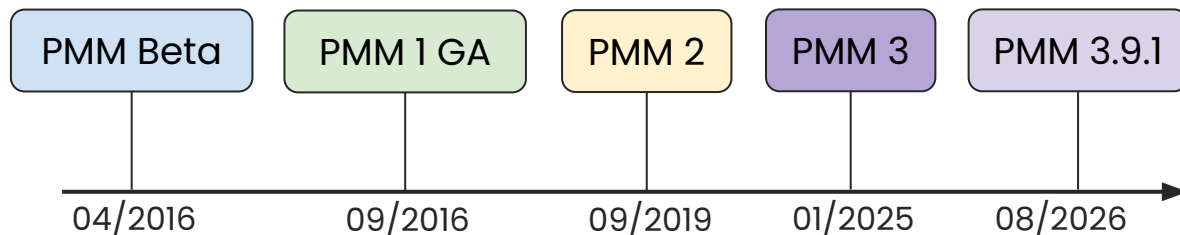


AGPL v3.0

Nando

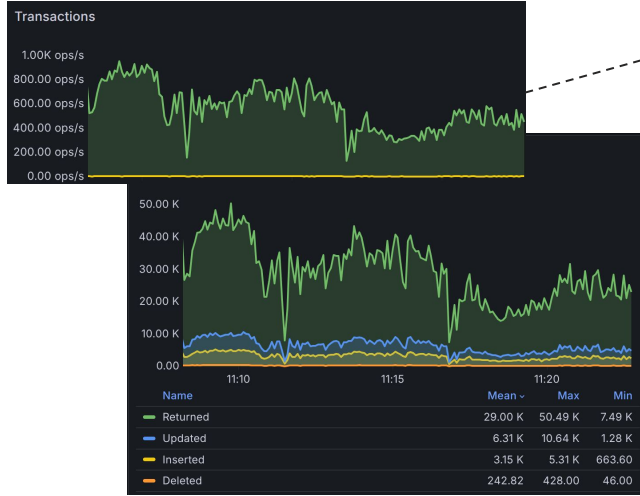
- Na Percona desde 2013
 - 8 anos Suporte
 - 4 anos Consultoria
- MySQL, PostgreSQL, K8s

Percona Monitoring and Management

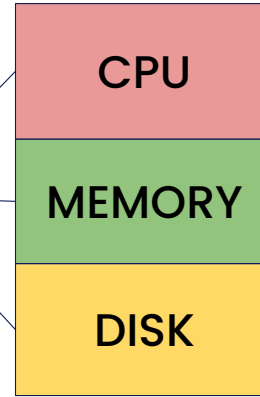


MySQL Tuning

MySQL tuning



MySQL™



Storage

► MySQL tuning

1 MySQL general configuration (my.cnf)

2 Schema optimization + query tuning

- Schema changes
- Query rewriting

Add/remove indexes

Changing data types

Installing PMM

▶ Installing PMM: server *Docker*

- Single script:

```
curl -fsSL https://www.percona.com/get/pmm | /bin/bash
```

- Manually:

```
docker pull percona/pmm-server:3
docker volume create pmm-data
docker run --detach --restart always --publish 443:8443 --volume pmm-data:/srv --name pmm-server
percona/pmm-server:3
```

▶ Installing PMM: client

- Database user:

```
CREATE USER 'pmm'@'localhost' IDENTIFIED BY 'secret' WITH MAX_USER_CONNECTIONS 10;  
GRANT SELECT, PROCESS, REPLICATION CLIENT, RELOAD ON *.* TO 'pmm'@'localhost';
```

- PMM agent, from Percona repository:

```
dnf install -y https://repo.percona.com/yum/percona-release-latest.noarch.rpm  
percona-release enable pmm3-client  
dnf install pmm-client
```

- Point the agent to the PMM server:

```
pmm-admin config --server-insecure-tls --server-url=https://admin:'admin'@10.128.0.123:443
```

- Configure the agent to monitor MySQL:

```
pmm-admin add mysql --username=pmm --password='secret' --socket=/var/run/mysqld/mysqld.sock  
--query-source=perfschema
```

MySQL Performance

Experiment

- Database: cloud VM with **32 GB** RAM, **8** vCPUs, 500 GB SSD, MySQL 8.4

```
sysbench /usr/share/sysbench/oltp_read_only.lua --db-driver=mysql  
--db-ps-mode=disable --mysql-ignore-errors=all --skip_trx=on --mysql-user=sysbench  
--mysql-password='secret' --mysql-db=sbtest --mysql-host=10.128.0.76 --tables=8  
--threads=64 --table_size=n --report-interval=1 --time=900 prepare
```

	<i>small</i>	<i>big</i>
--table_size (rows)	10000000	100000000
database size	~19 GB	~190 GB
table size	~2.3 GB	~23 GB



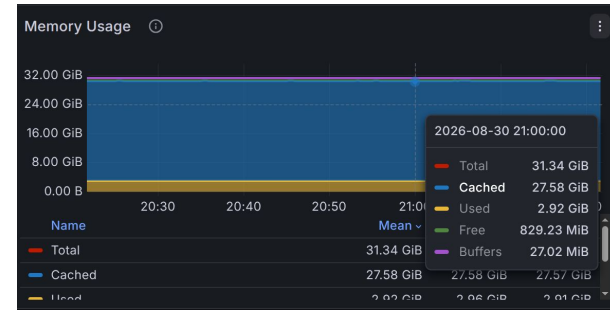
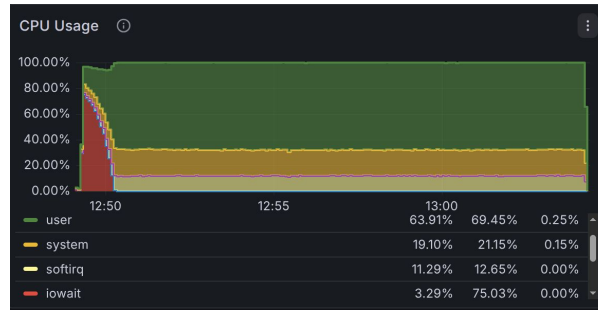
Experiment

Small dataset, default values, cold cache

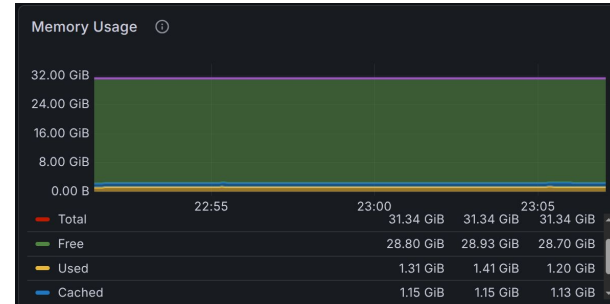
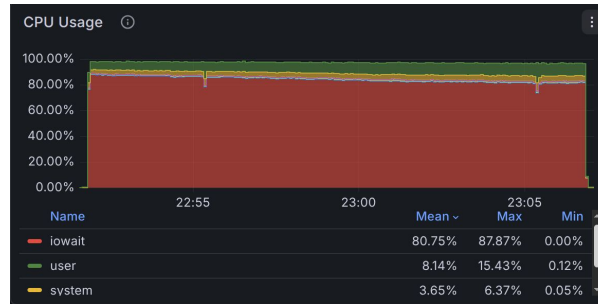
shared_buffers = 128MB

innodb_buffer_pool_size = 128 MB

PostgreSQL



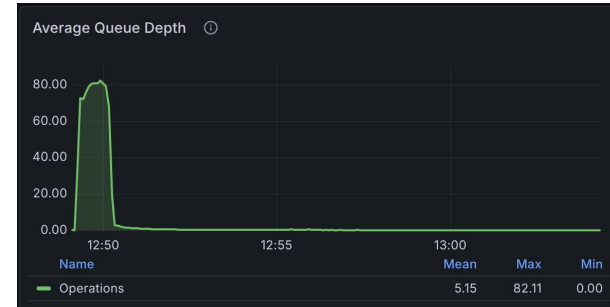
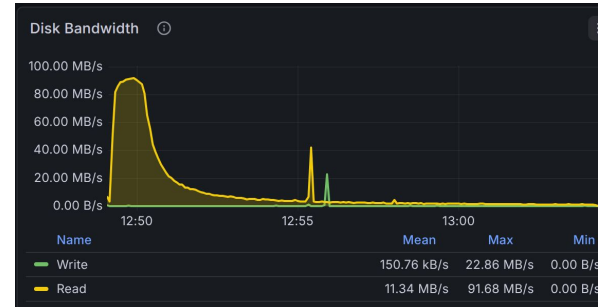
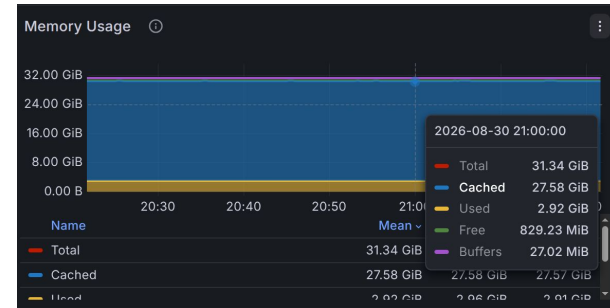
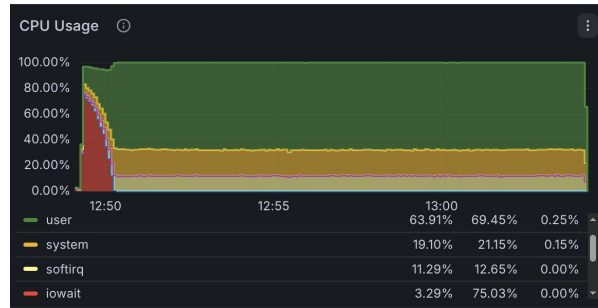
MySQL



Experiment

Small dataset, default values, cold cache

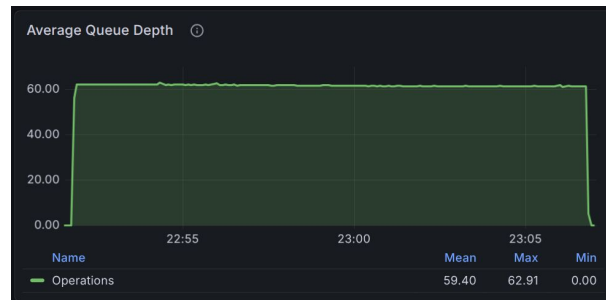
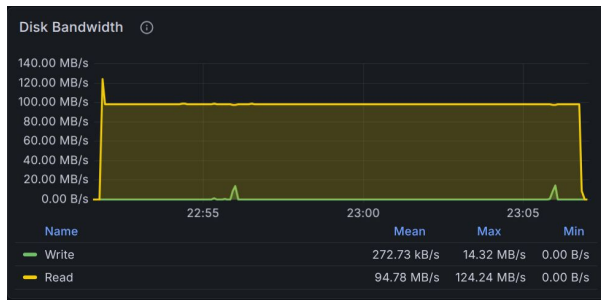
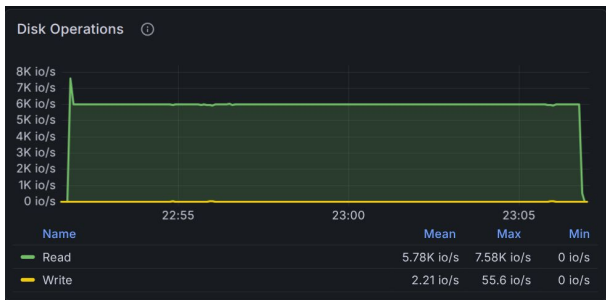
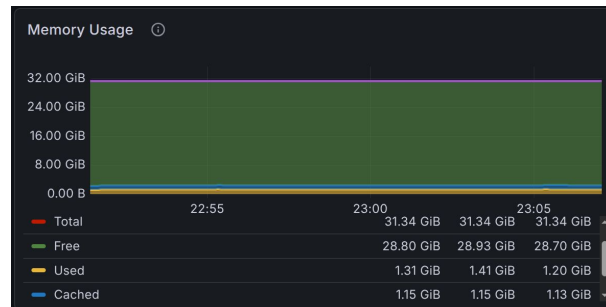
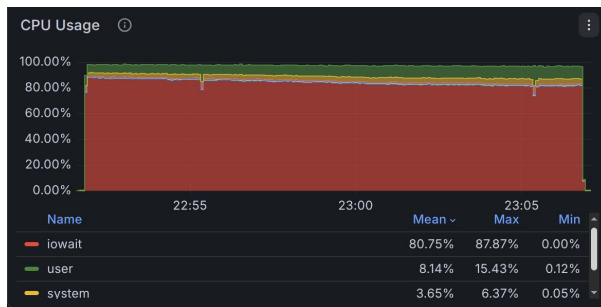
PostgreSQL



Experiment

Small dataset, default values, cold cache

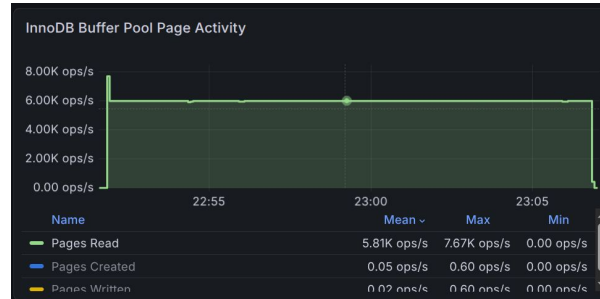
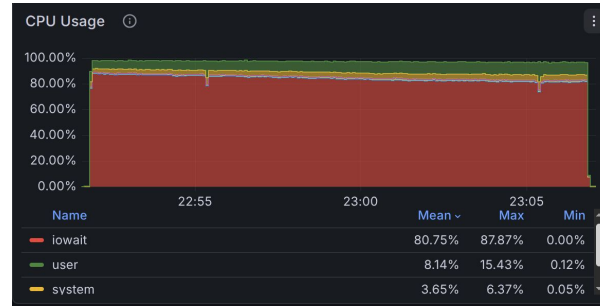
MySQL



Experiment

Small dataset, default values, cold cache

MySQL

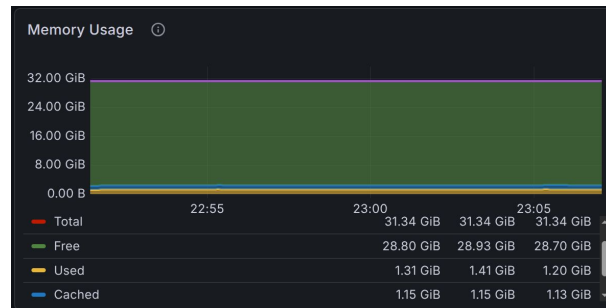
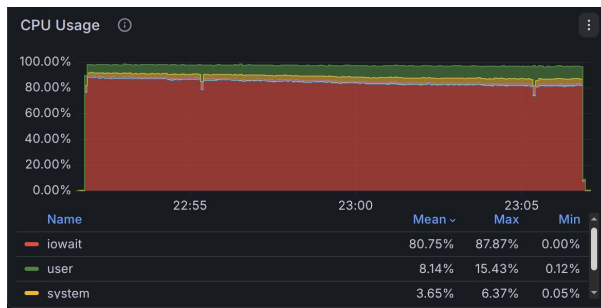




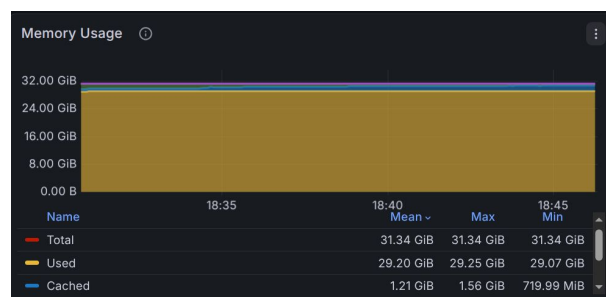
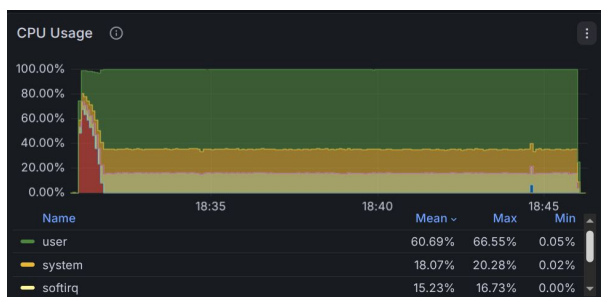
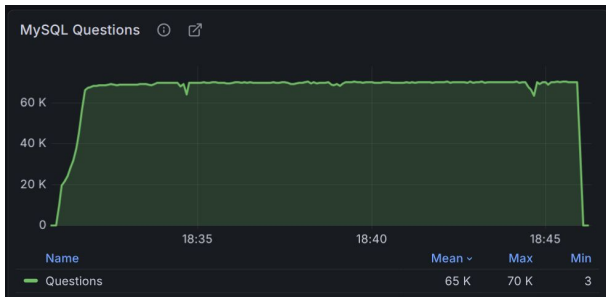
Experiment

Small dataset, untuned Vs tuned

MySQL (BP=128MB)



MySQL (BP=26GB)

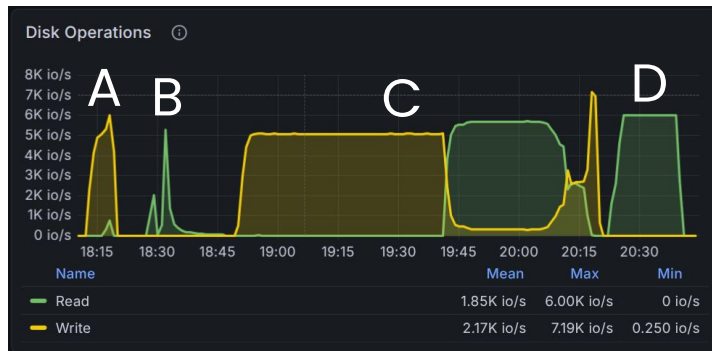
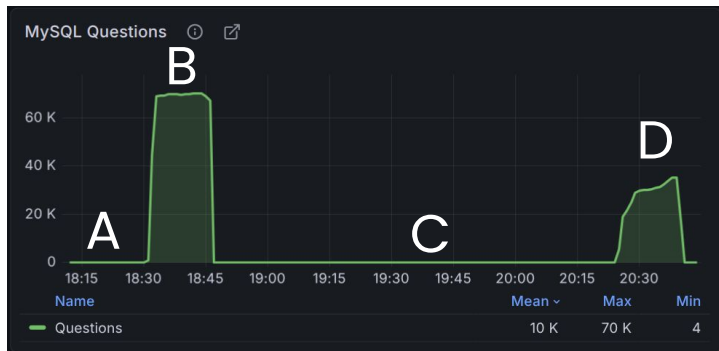


Experiment

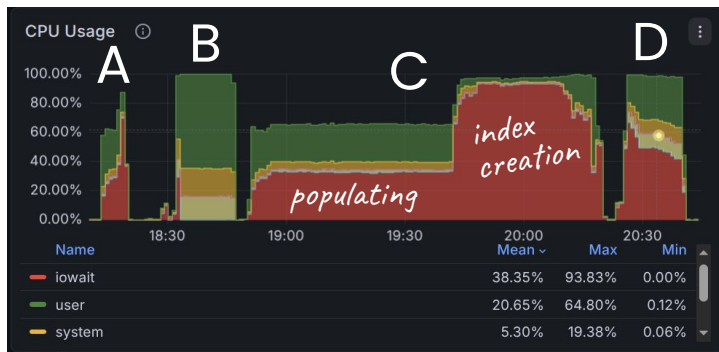
Database **tuned**:

- BP=26G
- redo logs=20G

- A) Small dataset, *prepare*
- B) Small dataset, *run*
- C) Big dataset, *prepare*
- D) Big dataset, *run*



IO

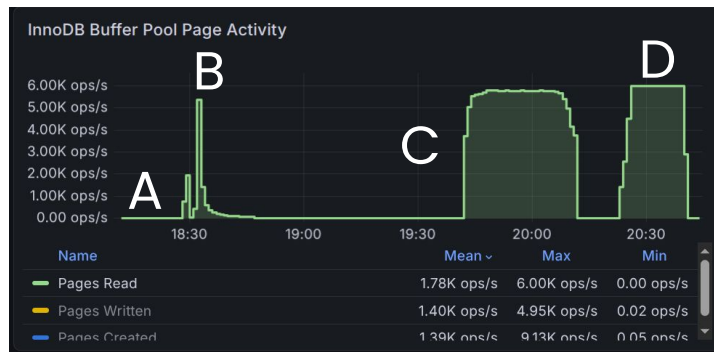
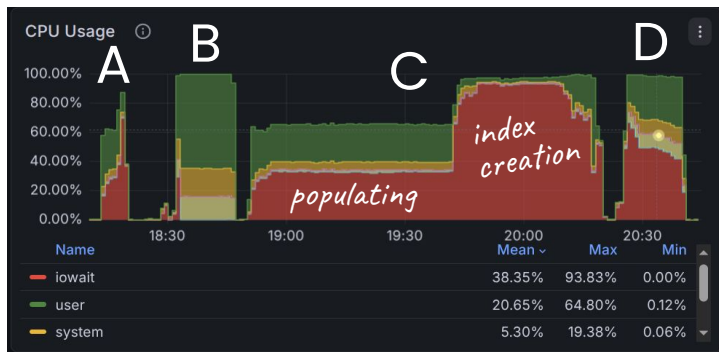
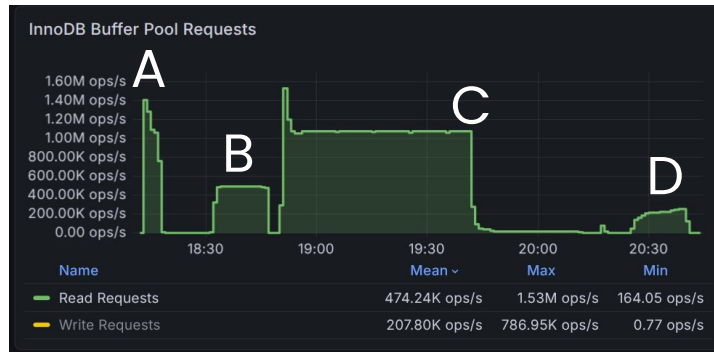
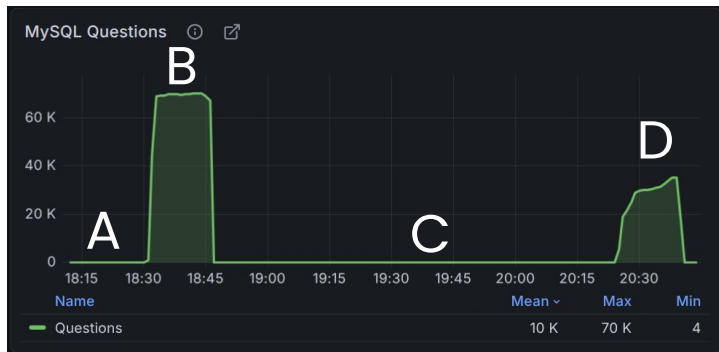


Experiment

Database **tuned**:

- BP=26G
- redo logs=20G

- A) Small dataset, *prepare*
- B) Small dataset, *run*
- C) Big dataset, *prepare*
- D) Big dataset, *run*



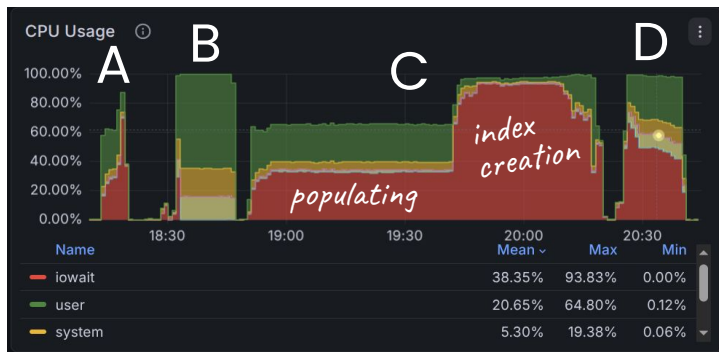
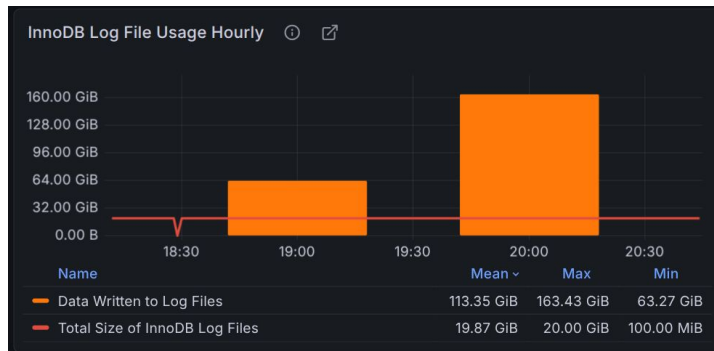
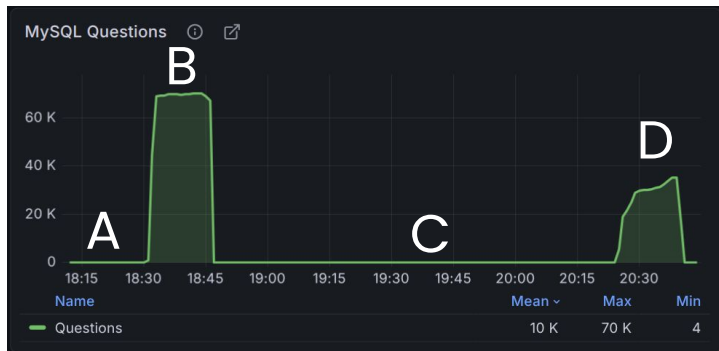
Does the
active data
fits in
memory?

Experiment

Database **tuned**:

- BP=26G
- redo logs=20G

- A) Small dataset, *prepare*
- B) Small dataset, *run*
- C) Big dataset, *prepare*
- D) Big dataset, *run*



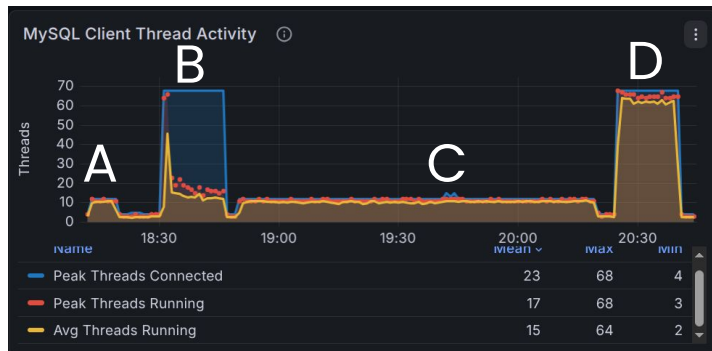
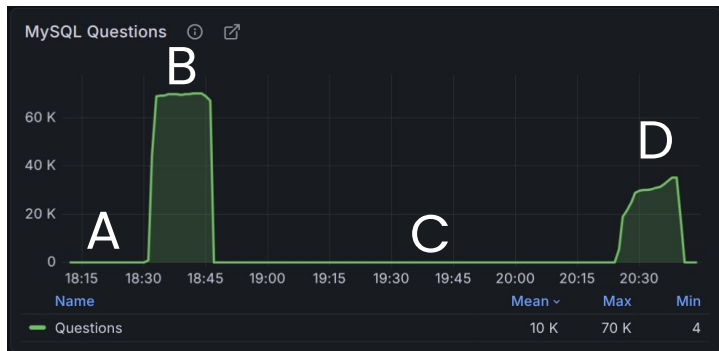
Checkpoint
performance

Experiment

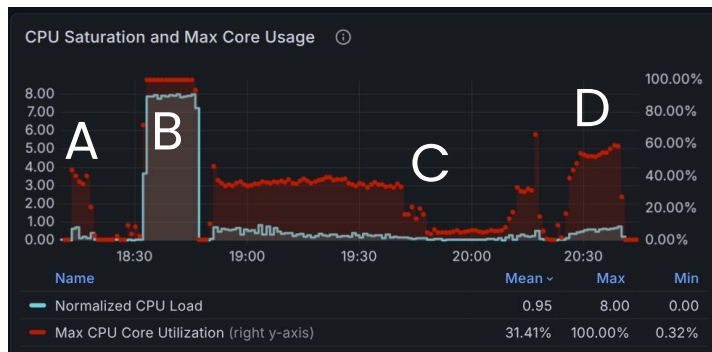
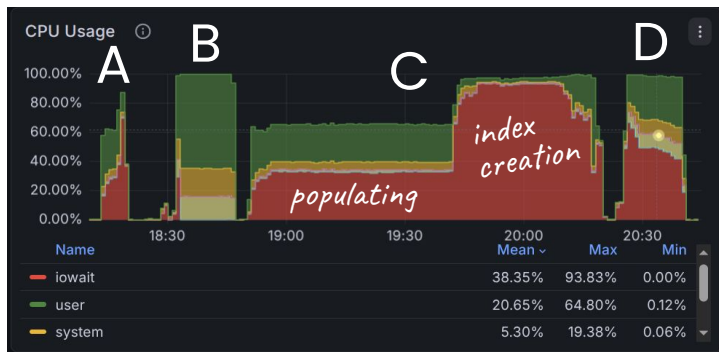
Database **tuned**:

- BP=26G
- redo logs=20G

- A) Small dataset, *prepare*
- B) Small dataset, *run*
- C) Big dataset, *prepare*
- D) Big dataset, *run*



connections



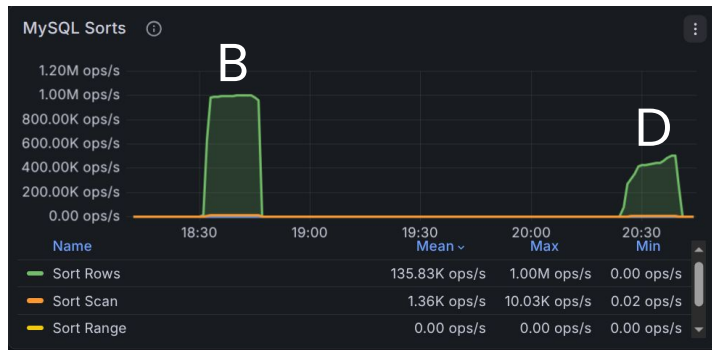
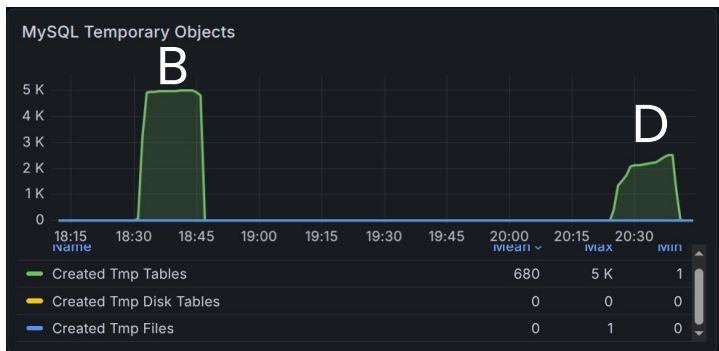
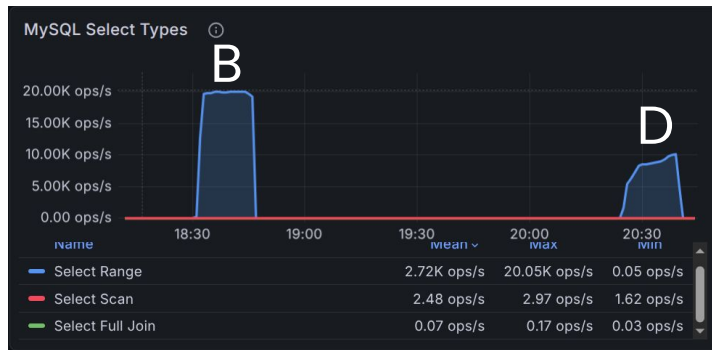
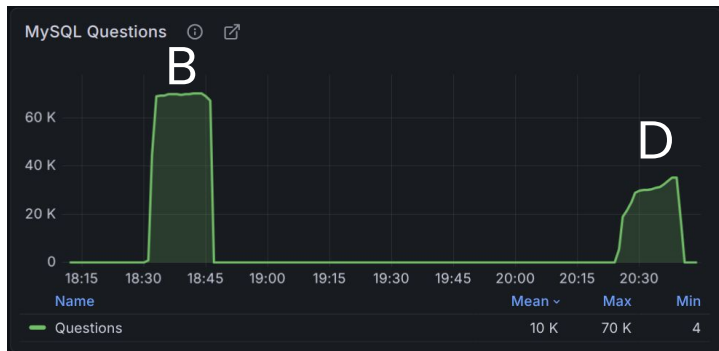
Experiment

Database **tuned**:

- BP=26G
- redo logs=20G

- A) Small dataset, *prepare*
- B) Small dataset, *run*
- C) Big dataset, *prepare*
- D) Big dataset, *run*

Workload character.



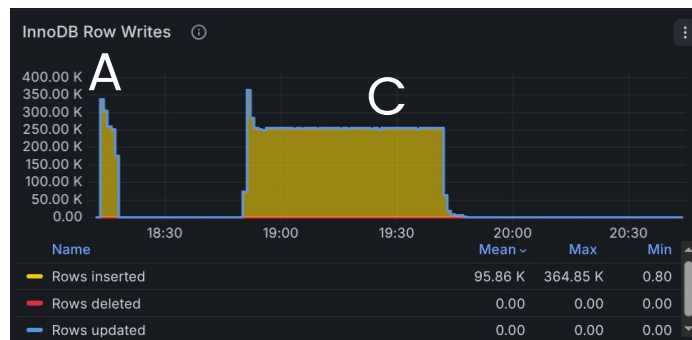
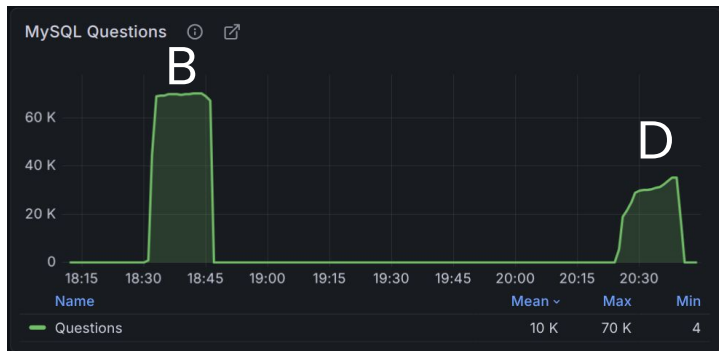
Experiment

Database **tuned**:

- BP=26G
- redo logs=20G

- A) Small dataset, *prepare*
- B) Small dataset, *run*
- C) Big dataset, *prepare*
- D) Big dataset, *run*

Workload character.

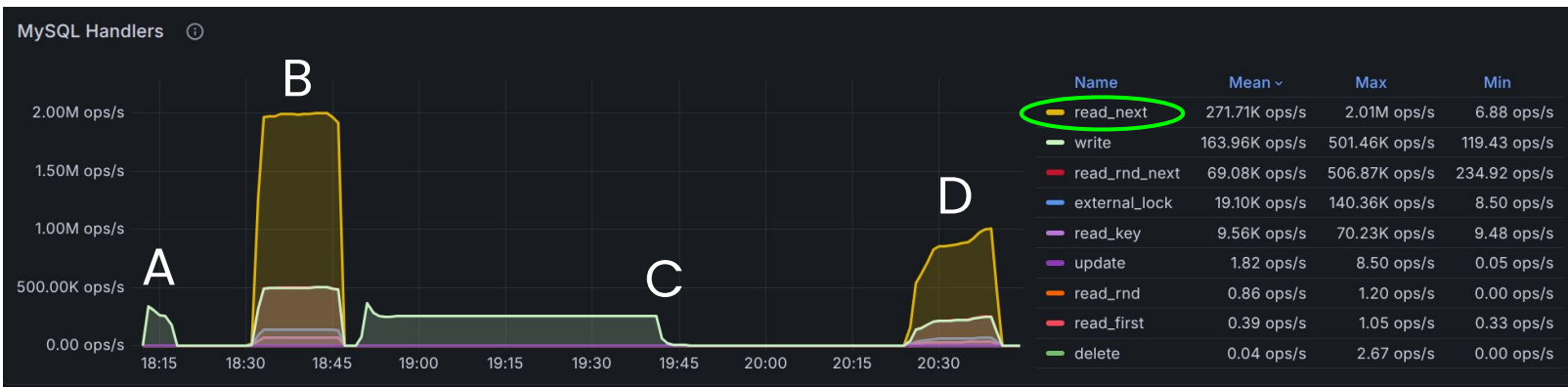


Experiment

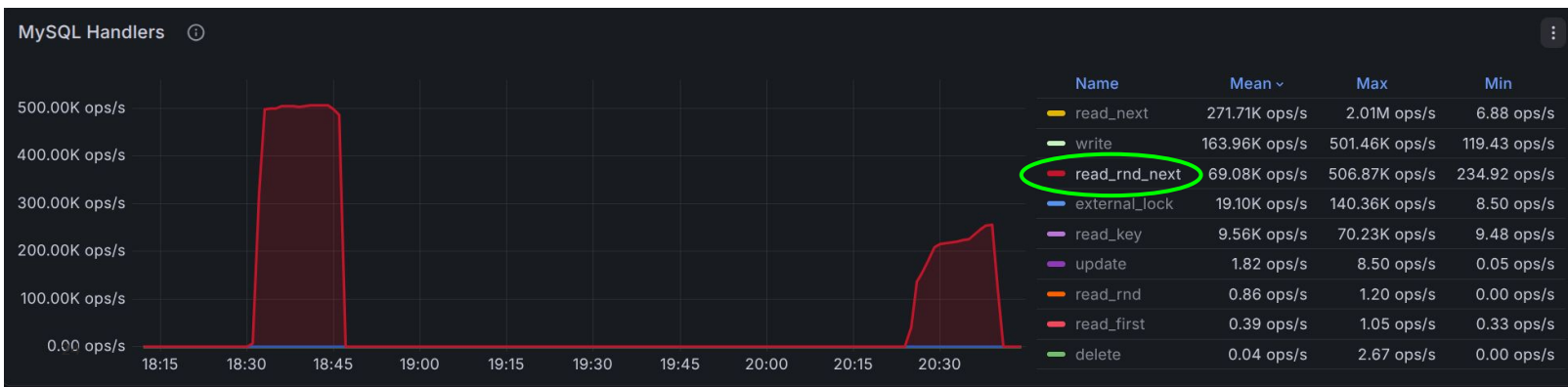
Database **tuned**:

- BP=26G
- redo logs=20G

- A) Small dataset, *prepare*
- B) Small dataset, *run*
- C) Big dataset, *prepare*
- D) Big dataset, *run*



Index usage
Vs
table scans



ERCONA®

Experiment

Database **tuned**:

- BP=26G
- redo logs=20G

- A) Small dataset, *prepare*
- B) Small dataset, *run*
- C) Big dataset, *prepare*
- D) Big dataset, *run*



Table cache

MySQL main dashboards

Other dashboards

QAN



Filters

Show Selected

Reset All

Filter by...

Schema

2

☐ sbtest

99.93%

☐ n/a

0.07%

Node Name

1

☒ nando-mys...

100%

Service Name

1

☒ 34895506d...

100%

Service Type

1

☐ mysql

100%

Node Type

1

☒ generic

100%

#

Query

Search by...

Q

Load

TOTAL

9.94 load

1

SELECT `c` FROM `sbtest2` WHERE `id` = ?

0.45 load

2

SELECT `c` FROM `sbtest5` WHERE `id` = ?

0.45 load

3

SELECT `c` FROM `sbtest6` WHERE `id` = ?

0.45 load

4

SELECT `c` FROM `sbtest1` WHERE `id` = ?

0.45 load

5

SELECT `c` FROM `sbtest7` WHERE `id` = ?

0.45 load

6

SELECT `c` FROM `sbtest8` WHERE `id` = ?

0.45 load

7

SELECT `c` FROM `sbtest3` WHERE `id` = ?

0.45 load

8

SELECT `c` FROM `sbtest4` WHERE `id` = ?

0.45 load

9

INSERT INTO `sbtest7` (`k`, `c`, `pad`) VALUES (.....

0.38 load

10

INSERT INTO `sbtest6` (`k`, `c`, `pad`) VALUES (.....

0.38 load

11

INSERT INTO `sbtest1` (`k`, `c`, `pad`) VALUES (.....

0.36 load

12

INSERT INTO `sbtest4` (`k`, `c`, `pad`) VALUES (.....

0.36 load

13

INSERT INTO `sbtest5` (`k`, `c`, `pad`) VALUES (.....

0.35 load

14

INSERT INTO `sbtest3` (`k`, `c`, `pad`) VALUES (.....

0.35 load

15

INSERT INTO `sbtest8` (`k`, `c`, `pad`) VALUES (.....

0.35 load

16

INSERT INTO `sbtest2` (`k`, `c`, `pad`) VALUES (.....

0.35 load

<

1

2

3

4

5

>

25 / page

1-25 of 124 items

Details Examples Explain Tables						
Close						
Query time distribution						
Other: 99.87%						
Metrics						
Metric	Rate/Second			Sum		Per Query Stats
Query Count ⓘ	839.41 QPS			7.79m	8.9% of total	1.00
Query Time ⓘ	0.45 load			1:09:57	4.55% of total	538.49 μs
Lock Time ⓘ	<0.01 (avg load)			5.58 sec	8.62% of total 0.13% of query time	<0.72 μs
Rows Sent ⓘ	839.59 (per sec)			7.80m	0.4% of total	1.00
Rows Examined ⓘ	839.48 (per sec)			7.80m	0.2% of total 1.00 per row sent	1.00
Metadata						
Name		Value				
Node name		nando-mysqlbrconf-db				
Node type		generic				
Schema		sbtest				
Service name		34895506d691-mysql				
Service type		mysql				

Details Examples Explain Tables
<pre> SELECT c FROM sbtest2 WHERE id = 4975611 </pre>

[Details](#)[Examples](#)[Explain](#)[Tables](#)[Close](#)

Classic

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	sbtest2	NULL	const	PRIMARY	PRIMARY	4	const	1	100	NULL

JSON

```
▼ "root" : {  
  ▼ "query_block" : {  
    ▼ "cost_info" : {  
      "query_cost" : "1.00"  
    }  
    "select_id" : 1  
    ▼ "table" : {  
      "access_type" : "const"  
      ▼ "cost_info" : {  
        "data_read_per_join" : "736"  
        "eval_cost" : "0.10"  
      }  
    }  
  }  
}
```

sbtest2

Table

```
CREATE TABLE `sbtest2` (
  `id` int NOT NULL AUTO_INCREMENT,
  `k` int NOT NULL DEFAULT '0',
  `c` char(128) NOT NULL DEFAULT '',
  `pad` char(60) NOT NULL DEFAULT '',
  PRIMARY KEY (`id`),
  KEY `k_2` (`k`)
) ENGINE=InnoDB AUTO_INCREMENT=100000001 DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci
```

Status

Name	Engine	Version	Row_format	Rows	Avg_row_length	Data_length	Max_data_length	Index_length	Data_free	Auto_increment	Create_time	Up
sbtest2	InnoDB	10	Dynamic	98757230	208	20561526784	0	0	3145728	100000001	2026-09-17T22:40:58Z	nul

Indexes

Table	Non_unique	Key_name	Seq_in_index	Column_name	Collation	Cardinality	Sub_part	Packed	Null	Inc
sbtest2	0	PRIMARY	1	id	A	90281024	null	null	NULL	BT
sbtest2	1	k_2	1	k	A	20722146	null	null	NULL	BT

17	SELECT DISTINCTROW `c` FROM `sbtest1` WHERE `i...` ①		
18	SELECT DISTINCTROW `c` FROM `sbtest3` WHERE `...` ①		
19	SELECT DISTINCTROW `c` FROM `sbtest7` WHERE `...` ①		

< 1 2 3 4 5 > 25 / page 1-25 of 124 items

Details Examples Explain Tables

```

SELECT
  DISTINCT c
FROM
  sbtest1
WHERE
  id BETWEEN 5614859
  AND 5614958
ORDER BY
  c

```

Details Examples Explain Tables Close

▼ Classic

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	sbtest1	NULL	range	PRIMARY	PRIMARY	4	NULL	100	100	Using where; Using temporary; Using filesort

API



→ ↺ docs.percona.com/percona-monitoring-and-management/3/api/index.html

← PERCONA MONITORING AND MANAGEMENT DOCUMENTATION

Percona Monitoring and Mana... 3.x (LATEST) ⚙️ 🔍 Search ✨ Ask Percona AI

Percona Monitoring and Management

Quickstart guide

Install >

Use >

Configure >

Administer >

Upgrade >

Uninstall >

Troubleshoot >

Release notes >

Reference >

Architecture >

PMM Architecture

Third-party components in PMM >

PMM components and versions

Data handling in PMM

PMM API >

About PMM API

Service accounts authentication

About PMM API

PMM Server provides a comprehensive REST API that enables you to monitor databases, manage resources, collect metrics, and automate PMM operations programmatically. The API supports endpoints for managing nodes, services, agents, alerting, backups, and other PMM components.

The [complete API documentation on Readme.io](#) includes detailed endpoint specifications, authentication methods, example requests, and response schemas.

Interactive API documentation

You can explore and test the API using the built-in [Swagger](#) UI, accessible at the `/swagger/` endpoint of your PMM Server. The Swagger interface allows you to:

- browse available API endpoints
- view detailed request and response schemas
- execute API requests directly from the browser
- test different parameters and see live responses

swagger /swagger.json Explore

PMM API public /swagger.json



Percona Monitoring and Management

Home page

MySQL

PostgreSQL

Operating system

All dashboards

Query Analytics (QAN)

Explore

PromQL builder

Alerts

Advisors

Inventory

Backups

Configuration

Users and access

Account

Help

Explore > Metrics

OutlineMetrics

SplitAdd<<⌚>>🔍↺

QueriesGraph

A (Metrics)

BuilderCode

Metrics browser >

```
avg by (service_name) (rate(mysql_global_status_innodb_buffer_pool_read_requests {service_name=~"34895506d691-mysql"}[1m]) or irate(mysql_global_status_innodb_buffer_pool_read_requests {service_name=~"34895506d691-mysql"}[5m]))
```

Options Legend: Read Requests Format: Time series Step: 1m Type: Both Exemplars: false

B (Metrics)

BuilderCode

Metrics browser >

```
avg by (service_name) (rate(mysql_global_status_innodb_buffer_pool_write_requests {service_name=~"34895506d691-mysql"}[1m]) or irate(mysql_global_status_innodb_buffer_pool_write_requests {service_name=~"34895506d691-mysql"}[5m]))
```

Options Legend: Write Requests Format: Time series Step: 1m Type: Both Exemplars: false

+ Add query

🕒 Query history

🔍 Query inspector

Graph

LinesBarsPointsStacked linesStacked bars

Time	Read Requests (Mil)	Write Requests (K)
18:20	0	0
18:25	1.4	0.7
18:30	0	0
18:40	0.5	0.5
18:50	1.5	0.8
19:00	1.1	0.6
19:45	0	0
20:00	0	0
20:10	0	0
20:20	0.1	0.1
20:30	0.2	0.2
20:40	0	0

Read RequestsWrite Requests



```
fernando@P14s:~$ curl -s --insecure --user 'admin:1234567890' 'https://34.172.181.245/prometheus/api/v1/query_range' --data-urlencode 'start=2026-09-17T21:10:07Z' --data-urlencode 'end=2026-09-17T23:44:53Z' --data-urlencode 'step=5s' --data-urlencode 'query=avg by (service_name) (rate(mysql_global_status_innodb_buffer_pool_read_requests{service_name=~"34895506d691-mysql"}[5s]))' | jq -r '.data.result[] | .metric.service_name as $service | .values[] | [$service, .[0], .[1]] | @tsv'
```

34895506d691-mysql	1789679487	106.2
34895506d691-mysql	1789679492	103.8
34895506d691-mysql	1789679497	22
34895506d691-mysql	1789679502	4.4
34895506d691-mysql	1789679507	17.6
34895506d691-mysql	1789679512	0
34895506d691-mysql	1789679517	1916.2
34895506d691-mysql	1789679522	0.2
34895506d691-mysql	1789679527	566187.8
34895506d691-mysql	1789679532	1759532.8
34895506d691-mysql	1789679537	1764694.4
34895506d691-mysql	1789679542	1750129.2
34895506d691-mysql	1789679547	1759162.8
34895506d691-mysql	1789679552	1565047.6
34895506d691-mysql	1789679557	1592896.8
34895506d691-mysql	1789679562	1539764.6
34895506d691-mysql	1789679567	1551454.4
34895506d691-mysql	1789679572	1523630.6
34895506d691-mysql	1789679577	1520062.2
34895506d691-mysql	1789679582	1397916.8
34895506d691-mysql	1789679587	1353102.8
34895506d691-mysql	1789679592	1373799
34895506d691-mysql	1789679597	1349698
34895506d691-mysql	1789679602	1337803.6
34895506d691-mysql	1789679607	1318700.2

Other features

Alerts

Learn more'. At the bottom, there are three buttons: '+ New alert rule from template', '+ New alert rule', and '+ New recording rule'."/>

Percona Monitoring and Management

Alerting > Alert rules

Alert rules

Rules that determine whether an alert will fire

Search by data sources ⓘ All data sources

Dashboard Select dashboard

State Rule type Health Contact point

Firing Normal Pending Recovering Alert Recording Ok No Data Error Choose a contact point

Search ⓘ Q Search

View as Grouped List State

You haven't created any rules yet

You can also define rules through file provisioning or Terraform [Learn more](#)

+ New alert rule from template + New alert rule + New recording rule

▶ Advisors

Percona Monitoring and Management

Advisors > Advisor Insights

Advisor Insights

Service Name	Fail Count by Severity
34895506d691-mysql	Warning 4 Notice 5

Advisors

Advisors > Advisor Insights

Advisor Insights

Failed Checks for service "34895506d691-mysql"

Check Name	Summary	Description	Severity	Details
mysql_security_password_... ↓	Passwords don't expire	Please consider imposing an expiry time for passwords by setting default_password_lifetime to a positive integer, indicating how many days can go by before the user having to renew their password; This will help prevent brute force attacks and risks of data leaks. Current settings = 0 ;	Warning	Read More
mysql_require_secure_tran... ↓	MySQL server allows unencrypted remote connections.	MySQL server allows unencrypted remote connections.	Warning	Read More
mysql_config_sql_mode ↓	Server is not configured to enforce data integrity	In order for maximum data integrity to be set, the server should have TRADITIONAL, STRICT_ALL_TABLES, STRICT_TRANS_TABLES configured in sql_mode. From these TRADITIONAL, STRICT_ALL_TABLES are missing.	Warning	Read More

Backups

Percona Monitoring and Management

Home page

MySQL

PostgreSQL

Operating system

All dashboards

Query Analytics (QAN)

Explore

Alerts

Advisors

Inventory

Backups

All backups

Scheduled backup jobs

Restores

Storage locations

Backup > Scheduled Backup Jobs

Scheduled Backup Jobs

Filter

Percona Monitoring and Management 3.x (LATEST)

Percona Monitoring and Management

MySQL backup prerequisites

Supported setups for MySQL backups

Create a MySQL backup

Restore a MySQL backup

Edit a scheduled backup

Delete a backup

6 Verify that a mysql system

7 Verify that MySQL uses /v

8 Grant pmm-agent read/wr

9 Install the latest versions of

version requirements, see

[xtrabackup](#) (incl

[qpress]]PERCONA

XtraBackup and M

When installing xtrabackup, U

version:

- MySQL 5.5, 5.6, 5.7 — PXB
- MySQL 8.0.0–8.0.33 — PXB
- MySQL 8.0.34+ — PXB 8.0.

Create scheduled backup

